Multimodal DuoAttention: Adapting DuoAttention for Efficient Video Question Answering

Rishi Dinesh
University of Toronto
rishidinesh@cs.toronto.edu

Navdeep Singh
University of Toronto
navdeep@cs.toronto.edu

Mihir Shah
University of Toronto
mihirshah@cs.toronto.edu

Abstract

Streaming Video Question Answering (VQA) with multimodal LLMs is expensive because each incoming frame adds visual tokens, increasing prefill cost and growing the KV cache with video length. DuoAttention [1] is a text-only inference method for long-context LLMs that learns, for each decoder attention head, whether it should act as a *retrieval head* with access to the full KV cache over the entire prefix, or as a *streaming head* that attends only to attention sink tokens and a short recent window using a much smaller KV cache. We extend this idea to video-language models with **Multimodal DuoAttention (MMDA)**, which learns this head partition on multimodal inputs and applies full attention only to retrieval-critical heads while running the remaining heads in streaming mode. On offline VQA benchmarks, MMDA largely preserves full-attention accuracy while reducing prefill memory and latency, and provides a controllable accuracy–KV budget tradeoff. On streaming VQA, MMDA is competitive with state-of-the-art efficiency methods while using much less memory and latency than full attention. These results suggest that head-level attention specialization is a practical systems mechanism for efficient long-context video understanding. Code and data are available here.

1 Background and Motivation

Modern video-language models represent a video as a sequence of sampled frames. Each frame is encoded into visual tokens by a vision encoder, projected into the language model's embedding space, and concatenated with text tokens for joint decoding. In this way, video understanding becomes long-context sequence modeling over a mixed stream of visual and textual tokens.

Offline and streaming video understanding place different systems demands on this pipeline. In offline settings, the full video is available before inference, so the system can sample frames globally, compress context with access to the whole clip, or adapt processing to the question. In streaming settings, frames arrive causally, future content is unavailable, and the model must remain ready to answer at arbitrary moments with low latency. This makes streaming especially relevant for real-time assistants and wearable systems that must continuously interpret an ongoing visual scene.

The main systems challenge is sustained context growth. As new frames arrive, the model accumulates more visual tokens, and cached decoder states grow accordingly. Caching past states avoids recomputation and supports fast responses when a question arrives, but naively caching all past visual context makes KV-cache memory grow with video stream length and eventually turns storage into the main bottleneck.

Recent streaming video methods improve efficiency through intelligent visual token compression, memory management, and KV-cache reduction techniques. However, these methods mostly decide which tokens, memories, or KV states to keep, compress, or retrieve; they do not ask whether decoder heads themselves use context differently and can therefore be managed asymmetrically.

DuoAttention [1] is a recent paper that makes this question explicit in text-only LLMs. It shows that attention heads are functionally asymmetric: a small subset acts as "retrieval heads" that benefit from full-context access, while most act as "streaming heads" that rely mainly on recent tokens and attention sinks. To identify these head types, DuoAttention trains on a synthetic needle-in-a-haystack text retrieval task where the model must recover information from far back in the sequence, and uses this to learn per-head preference for full versus sink-plus-recent attention.

This hybrid design reduces pre-filling and decoding memory and latency with little loss in long-context accuracy, suggesting that head-level specialization is itself a useful systems lever for efficient inference. However, DuoAttention is developed and validated only for text. It does not test whether the same retrieval-versus-streaming head specialization persists in multimodal VideoQA, where long video streams introduce large visual prefixes and retrieval may depend on visual-text interactions. That gap motivates the present work.

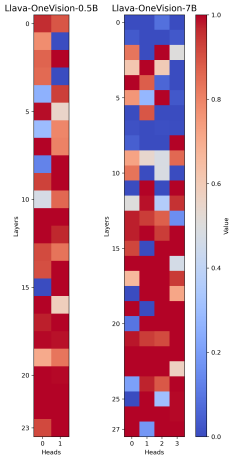

Figure 1: Optimized gate values of LLaVA-OneVision models. Retrieval heads have higher scores.

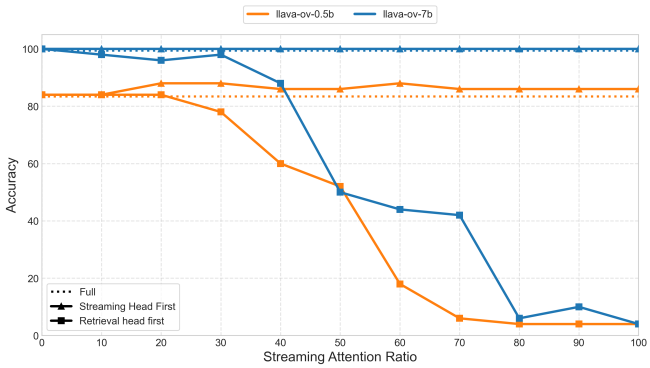

Figure 2: The impact of limiting attention to the sink and recent tokens on video needle-in-a-haystack task performance: modifying retrieval heads severely damages performance, while constraining streaming heads has minimal to no impact.

2 Goal

We ask whether the retrieval-versus-streaming head specialization identified by DuoAttention [1] in text-only LLMs also appears in video-capable multimodal models, and whether exploiting it can reduce the cost of long-video question answering. To study this, we adapt DuoAttention to the multimodal setting as **Multimodal DuoAttention (MMDA)**, a decoder-side KV-cache management method for video-language models. Our hypothesis is as follows:

> A small subset of decoder heads in a video-capable multimodal model are responsible for long-range visual-text retrieval, and preserving full context only for these heads will maintain most VideoQA performance while significantly reducing KV-cache memory and inference latency.

Our contributions are threefold:

- We propose a **synthetic video needle-in-a-haystack generator** for multimodal head identification. Starting from ordinary videos, we insert random subtitle-based needles into selected frames and train the model to recover them in the answer, creating controlled retrieval examples that force the model to access sparse, distant visual evidence and thereby expose which decoder heads must retain full context during training.
- We **adapt DuoAttention to a video-language model**, focusing on the LLaVA-OneVision family. We freeze the backbone and learn head-level gates only in the language decoder, partitioning decoder heads into *retrieval heads*, which retain full multimodal KV history, and *streaming heads*, which attend only to sink tokens and a recent local window. We refer to this multimodal adaptation as MMDA.
- We **evaluate MMDA on offline and online VideoQA** to measure the tradeoff between efficiency and accuracy, reporting answer quality together with KV-cache memory usage and inference latency.

In summary, our goal is to test whether head-level specialization is a useful systems lever for efficient multimodal long-context inference.

3 Related Work

We focus our literature survey on streaming VQA as we think MMDA will be most useful in this setting. Recent work tackles the systems challenges of streaming VideoQA at three main points in the pipeline: by building compact memories over time, by reducing visual tokens before they reach the decoder, or by directly limiting the decoder KV cache.

Memory management. VideoStreaming [2] processes videos clip by clip, carries forward a condensed memory, and then selects a small set of query-relevant memories at question time. Flash-VStream [3] uses a two-level memory, with a compact long-term store for global context and a larger auxiliary store for finer detail. Moving to coarser temporal units, Vista [4] groups frames into scenes, stores scene tokens, and recalls only relevant scenes for later questions, while Event-VStream [5] writes one

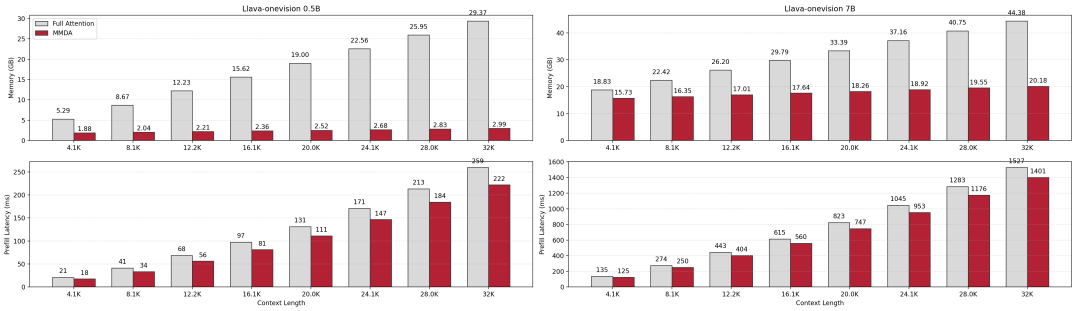

Figure 3: Prefill latency and memory usage of MMDA compared to full attention across varying context sizes.

compact memory per detected event and updates memory only at event boundaries. These methods make long streams tractable by replacing raw frame history with clip-, scene-, or event-level memory, but they do not change how the decoder attends over its cached context.

Visual token compression. Another common strategy is to reduce visual tokens before they create a large KV cache. TimeChat-Online [6] drops tokens that change little across adjacent frames, based on the observation that much of a video stream is visually redundant. StreamingTOM [7] keeps a fixed number of tokens per frame using change and saliency signals, and quantizes the retained KV states into a 4-bit online memory, which reduces storage cost while preserving on-demand access to past context. STC [8] goes one step earlier by caching ViT features for similar frames and pruning tokens before the language model, reducing both vision-side and language-side cost. FluxMem [9] removes repeated content across nearby frames and also merges repetitive regions within each frame. Together, these methods reduce future KV growth indirectly by sending fewer visual tokens forward, but they still treat the problem mainly as token compression rather than decoder-side attention design.

Direct KV-cache management. A more direct line of work treats the KV cache itself as the main bottleneck. ReKV [10] stores past video KV caches outside active GPU memory and retrieves only the parts that appear relevant when a question arrives. StreamMem [11] compresses streaming visual KVs online into a fixed-size memory using a generic template user query, while InfiniPot-V [12] enforces a hard memory cap by removing temporally redundant entries and keeping more informative ones. LiveVLM [13] builds a streaming-oriented KV cache for short-term and long-term retrieval, while StreamKV [14] adds segment summaries and layer-adaptive compression to unify retrieval and compression in one module. HERMES [15] treats the cache as hierarchical memory across layers and timesteps, and StreamingVLM [16] keeps only attention sinks together with short recent vision context and a longer text window. These methods directly control KV growth, but they mainly decide which cached entries to keep, compress, or retrieve, rather than whether different decoder heads need different amounts of history.

DuoAttention and the remaining gap. Closest to our work, DuoAttention [1] argues that decoder heads in long-context language models do not all need the same history: retrieval heads require global access, while streaming heads mainly use recent tokens and attention sinks, so preserving full KV history only for a subset of heads can reduce memory and latency while maintaining long-context ability. However, DuoAttention is developed for text, not for multimodal VideoQA, where the cache is dominated by visual tokens and long-range retrieval may depend on video-text interactions. Existing video methods mainly compress visual inputs, build higher-level memories, or prune and retrieve KV entries, but they do not test whether multimodal decoder heads themselves separate into global-retrieval and local-streaming roles. MM-DuoAttention (MMDA) addresses that gap by applying head-level attention partitioning directly to video-conditioned multimodal decoding.

4 Methodology

4.1 Dataset

To identify which decoder heads require global multimodal context, we train MMDA on a synthetic video needle-in-a-haystack retrieval task constructed from ordinary source videos. For each example, the generator samples frames from a real video, inserts subtitle-style secret words into a small number of selected frames, and asks the model to recover those words in order. The untouched video content acts as the haystack, while the subtitle-injected frames act as the needles. In our implementation, the subtitles follow the VNBench rendering style [17] with white text displayed on a full-width gray bar near the bottom of the frame, using templates such as "The first secret word is: {word}". More details are in Appendix A.

The result is a controlled multimodal retrieval problem: the model cannot answer correctly unless it recovers sparse and potentially distant evidence from the visual stream. Because the secret words are random, they cannot be inferred from prior world knowledge, which helps isolate long-range retrieval from general semantic reasoning.

This synthetic setup is a useful probe for retrieval heads because it creates explicit retrieval demands inside a multimodal sequence. Heads that genuinely need full attention should remain important when the answer depends on distant subtitle events, while heads that mostly support local composition should be able to operate accurately with only a recent window and attention sinks.

4.2 Training

We adapt DuoAttention to the multimodal setting by freezing the backbone and learning only one gate per KV head per decoder layer. Each gate determines whether a head behaves more like a full-context retrieval head or a sink-plus-recent streaming head.

Training uses a teacher–student setup. Each batch is duplicated: the teacher uses standard full causal attention, while the student mixes full attention and streaming attention on a per-head basis. For a head with gate g_h, the student output is

$$\mathbf{o}_h^{\text{mixed}} = (1 - g_h)\, \mathbf{o}_h^{\text{stream}} + g_h\, \mathbf{o}_h^{\text{full}}.$$

Here, $\mathbf{o}_h^{\text{full}}$ is the output under ordinary causal attention over the full multimodal prefix, and $\mathbf{o}_h^{\text{stream}}$ is the output under a streaming mask that keeps only attention sinks and a recent local window.

The student is trained to match the teacher only on the answer-token hidden states. If $H_{\text{ans}}^{\text{teacher}}$ and $H_{\text{ans}}^{\text{student}}$ denote the teacher and student hidden states at answer positions, the loss is

$$\mathcal{L} = \text{MSE}\big(H_{\text{ans}}^{\text{student}},\, H_{\text{ans}}^{\text{teacher}}\big) + \lambda \sum_h |g_h|.$$

where the first term is the distillation loss that matches the student to the teacher only on the answer-token hidden states, and the second term is an L_1 regularization term on the gates to encourage a sparse set of full-attention heads. Heads with gates near 1 are interpreted as retrieval heads, and heads with gates near 0 are interpreted as streaming heads. Figure 1 shows the final optimized gate values after training. More details are in Appendix B.

Inference and deployment. After training, the learned gate values are binarized using either a threshold or a target sparsity level to assign each head to full attention or streaming attention. Here, sparsity denotes the fraction of heads assigned to streaming attention, so 75% sparsity means heads are ranked by their learned gate scores and the bottom 75% fraction are assigned to streaming attention, while the remaining heads retain full attention. Before deployment, the query, key, and value projections are reordered so that retrieval heads and streaming heads are packed into contiguous groups, which makes the mixed attention pattern more efficient to execute. During decoding, each layer maintains two KV caches: a full cache for retrieval heads and a bounded sink-plus-recent cache for streaming heads. Attention is computed separately for the two groups, then the outputs are concatenated for the final projection.

4.3 Validation

We developed a counterfactual stress test to validate MMDA and evaluated it on VNBench [17], a benchmark that contains video needle-in-a-haystack tasks, using exact-match accuracy to measure performance. Starting from an all-full baseline, we binarize the learned head scores into retrieval and streaming head groups, then progressively convert an increasing fraction of one group of heads from full attention to sink-plus-recent streaming attention while keeping all other heads full. We evaluate two settings: a **streaming-first** sweep, which sets streaming attention to identified streaming heads, and a **retrieval-first** sweep, which sets streaming attention to identified retrieval heads.

Figure 2 shows that the two sweeps separate clearly for both LLaVA-OneVision-0.5B and 7B. When the heads identified as streaming heads are progressively constrained, accuracy remains close to the full-attention baseline. In contrast, when the heads identified as retrieval heads are progressively constrained, accuracy drops sharply and eventually falls to very low levels. This asymmetry validates our MMDA training strategy by showing that it correctly identifies streaming and retrieval heads.

5 Results and Observations

5.1 Efficiency Results

Figure 3 shows that MMDA substantially reduces prefill memory and latency relative to full attention for both LLaVA-OneVision-0.5B and 7B, with the gap widening as context length increases. MMDA reduces KV cache memory by up to **9.8× on 0.5B** and **2.2× on 7B** at 32K context, with near-constant memory for streaming heads. Prefill latency improvements are moderate (8–14%) since prefill remains compute-bound. MMDA also remains usable beyond the context lengths where full attention runs out of

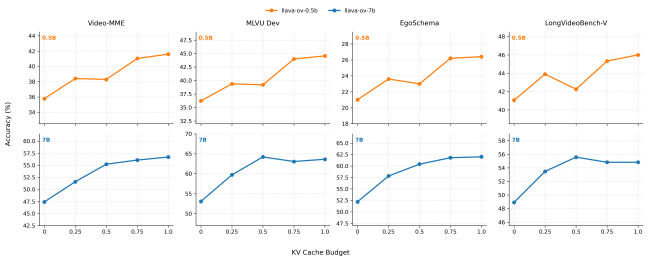

Figure 4: Offline benchmark accuracy across MMDA sparsity levels (with sink=256 & recent=512). MMDA provides a favorable accuracy-budget tradeoff over a range of cache budgets.

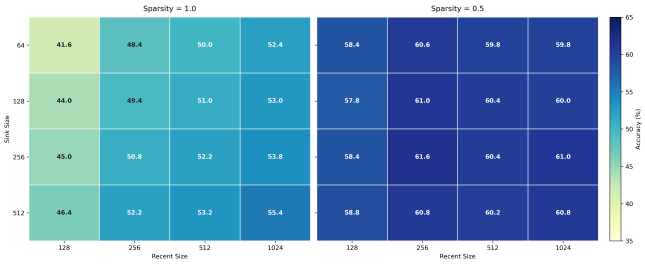

Figure 5: EgoSchema accuracy (for LLaVA-OV-7b) under varying sink and recent window sizes, comparing full streaming attention with MMDA at fixed sparsity (s=0.5). MMDA preserves task performance much better as the streaming window is reduced.

memory, continuing to operate at those lengths with much lower memory usage, thereby extending the model's practical context range. This advantage should become even more important as VLMs move beyond today's 32K-scale settings to 1M+ scale, where naive full-attention prefilling becomes increasingly impractical.

5.2 Offline Benchmark Results

Table 1 shows that MMDA largely preserves offline VideoQA performance relative to full attention across MLVU, Video-MME, EgoSchema, and LongVideoBench-V. Combined with the efficiency gains in Figure 3, this indicates that MMDA provides a **favorable memory-performance tradeoff**, retaining most benchmark accuracy while being faster and using much less memory than full attention.

Table 1: Performance comparison of LLaVA-OV models under Full Attention and MMDA

Model	Method	MLVU	Video-MME	EgoSchema	LongVideoBench-V
LLaVA-OV-0.5B	Full	44.57	41.59	26.40	46.00
	MMDA	39.21	38.30	24.00	42.26
LLaVA-OV-7B	Full	63.63	56.70	62.00	54.82
	MMDA	64.22	55.22	61.60	55.57

5.3 Controllable KV-Cache Tradeoffs

Figure 4 shows that sparsity is an effective first lever for trading KV-cache cost against accuracy: across both model sizes, MMDA retains most benchmark performance over a wide range of cache budgets, making sparsity a practical knob for selecting a favorable memory-performance operating point. Figure 5 shows that, beyond sparsity, the sink and recent window sizes provide an additional complementary lever for reducing memory footprint: even as these streaming-attention budgets are reduced, MMDA preserves task (EgoSchema) accuracy much better than fully streaming attention. Together, these experiments show that MMDA exposes multiple controllable knobs for lowering KV storage while maintaining strong task performance.

5.4 Online (Streaming) Benchmark Results

On real streaming video benchmarks, MMDA is the strongest overall method for the 0.5B model. Across both RVS-Ego and RVS-Movie, it improves over Full Attention while using substantially less GPU memory and lower latency, showing that selective full attention on a small subset of retrieval-critical heads is more effective than keeping full attention everywhere.

We also compare MMDA against two strong baselines with different efficiency strategies: ReKV, which retrieves relevant cache entries, and StreamingTom, which compresses tokens. The results show that MMDA is competitive with both approaches and offers a particularly strong balance of accuracy, speed, and memory efficiency. In particular, while StreamingTom reaches the lowest memory footprint, it does so with weaker overall task performance, whereas MMDA stays on a better quality-efficiency frontier. For the 7B model, due to hardware limitations, we could not run StreamingTom or RVS-Ego experiments, but the available results are still encouraging: Full Attention becomes infeasible (because of Out of GPU Memory), while MMDA remains runnable, demonstrating that the method scales to a larger model in settings where the standard baseline fails.

Table 2: Streaming VQA results for LLaVA-OneVision. We report accuracy, answer latency (secs/answer), and peak GPU memory (GB).

Model	Method	RVS-Ego			RVS-Movie		
		Accuracy↑	Latency↓	GPU↓	Accuracy↑	Latency↓	GPU↓
LLaVA-OV-0.5B	Full Attention	0.688	1.77	15.02	0.767	1.34	9.20
	MMDA	**0.740**	**0.33**	4.83	**0.777**	**0.33**	3.49
	ReKV	0.735	0.41	3.13	0.764	0.63	3.18
	MMDA + ReKV	0.737	0.59	3.15	0.768	0.89	3.21
	StreamingTom	0.735	0.97	**2.25**	0.756	0.99	**2.08**
	MMDA + StreamingTom	0.676	2.38	2.29	0.667	2.05	2.13
LLaVA-OV-7B	Full Attention	**OOM**	**OOM**	**OOM**	–	–	–
	MMDA	0.717	0.99	23.4	–	–	–
	ReKV	**0.736**	2.42	**21.9**	–	–	–
	MMDA + ReKV	0.730	**2.02**	**21.9**	–	–	–

Finally, the combination experiments show that efficient long-context methods are not automatically compositional. MMDA + ReKV does not provide meaningful additional memory savings and increases latency relative to using ReKV alone, suggesting overlapping benefits rather than true complementarity.

MMDA + StreamingTom performs worse as well, likely because both methods aggressively discard context in different ways, which interferes with the model's ability to preserve the temporal and semantic information needed for video understanding. Taken together, these results indicate that MMDA is most effective as a standalone attention strategy, and that combining multiple efficiency mechanisms requires careful joint design rather than simple stacking methods.

6 Takeaways

6.1 Key Learnings

Through this project, we learned that not all attention heads in an LLM or VLM contribute equally or perform the same function. Inspired by mechanistic interpretability work [18] [19] showing that attention heads can specialize into distinct roles, we tested whether the retrieval-versus-streaming specialization observed by DuoAttention [1] in long-context language models also transfers to multimodal video-language models. Our results show that it does. The key insight from this project is that knowledge of how attention heads behave non-uniformly can be turned into a practical systems mechanism: instead of allocating compute and KV-cache budget uniformly across all heads, we can allocate it based on function.

6.2 What worked and did not work

MMDA successfully translated this insight into a multimodal setting. The learned head partition showed that a small set of decoder heads remains retrieval-critical, while many other heads can operate effectively with sink-plus-recent streaming attention. This yields large memory savings relative to full attention while preserving most task performance on offline and online VideoQA benchmarks.

Just as importantly, once this specialization was identified, it opened up additional ways to push efficiency further: sparsity, sink size, and recent-window size all became useful knobs for squeezing more memory savings from the streaming heads, as long as the retrieval heads retained enough budget.

On the other hand, combining MMDA with existing efficiency methods did not yield additional gains. As shown in Table 2, composing MMDA with ReKV or StreamingTom hurt rather than helped, suggesting that stacking independent context-reduction strategies can interfere rather than complement.

7 Conclusion

The main takeaway from this project is that multimodal efficiency should not be framed only as a token compression or memory retrieval problem, as prior work did. It can also be framed as a head specialization problem. MMDA shows that decoder heads in VLMs can be managed asymmetrically based on their role, and that this yields substantial KV-cache savings with limited accuracy loss. More broadly, this project suggests that mechanistic insights about transformer internals can be directly useful for systems design, not just for interpretability.

References

[1] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024.

[2] Rui Qian, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Shuangrui Ding, Dahua Lin, and Jiaqi Wang. Streaming long video understanding with large language models. *Advances in Neural Information Processing Systems*, 37:119336–119360, 2024.

[3] Haoji Zhang, Yiqin Wang, Yansong Tang, Yong Liu, Jiashi Feng, and Xiaojie Jin. Flash-vstream: Efficient real-time understanding for long video streams. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 21059–21069, 2025.

[4] Haocheng Lu, Nan Zhang, Wei Tao, Xiaoyang Qu, Guokuan Li, Jiguang Wan, and Jianzong Wang. Vista: Scene-aware optimization for streaming video question answering under post-hoc queries. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 7539–7547, 2026.

[5] Zhenghui Guo, Yuanbin Man, Junyuan Sheng, Bowen Lin, Ahmed Ahmed, Bo Jiang, Boyuan Zhang, Miao Yin, Sian Jin, Omprakash Gnawal, et al. Event-vstream: Event-driven real-time understanding for long video streams. *arXiv preprint arXiv:2601.15655*, 2026.

[6] Linli Yao, Yicheng Li, Yuancheng Wei, Lei Li, Shuhuai Ren, Yuanxin Liu, Kun Ouyang, Lean Wang, Shicheng Li, Sida Li, et al. Timechat-online: 80% visual tokens are naturally redundant in streaming videos. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 10807–10816, 2025.

[7] Xueyi Chen, Keda Tao, Kele Shao, and Huan Wang. Streamingtom: Streaming token compression for efficient video understanding. *arXiv preprint arXiv:2510.18269*, 2025.

[8] Yiyu Wang, Xuyang Liu, Xiyan Gui, Xinying Lin, Boxue Yang, Chenfei Liao, Tailai Chen, and Linfeng Zhang. Accelerating streaming video large language models via hierarchical token compression. *arXiv preprint arXiv:2512.00891*, 2025.

[9] Yiweng Xie, Bo He, Junke Wang, Xiangyu Zheng, Ziyi Ye, and Zuxuan Wu. Fluxmem: Adaptive hierarchical memory for streaming video understanding. *arXiv preprint arXiv:2603.02096*, 2026.

[10] Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Tao Zhong, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, and Hao Jiang. Streaming video question-answering with in-context video kv-cache retrieval. *arXiv preprint arXiv:2503.00540*, 2025.

[11] Yanlai Yang, Zhuokai Zhao, Satya Narayan Shukla, Aashu Singh, Shlok Kumar Mishra, Lizhu Zhang, and Mengye Ren. Streammem: Query-agnostic kv cache memory for streaming video understanding. *arXiv preprint arXiv:2508.15717*, 2025.

[12] Minsoo Kim, Kyuhong Shim, Jungwook Choi, and Simyung Chang. Infinipot-v: Memory-constrained kv cache compression for streaming video understanding. *arXiv preprint arXiv:2506.15745*, 2025.

[13] Zhenyu Ning, Guangda Liu, Qihao Jin, Wenchao Ding, Minyi Guo, and Jieru Zhao. Livevlm: Efficient online video understanding via streaming-oriented kv cache and retrieval. *arXiv preprint arXiv:2505.15269*, 2025.

[14] Yilong Chen, Xiang Bai, Zhibin Wang, Chengyu Bai, Yuhan Dai, and Ming Lu. Streamkv: Streaming video question-answering with segment-based kv cache retrieval and compression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 3120–3128, 2026.

[15] Haowei Zhang, Shudong Yang, Jinlan Fu, See-Kiong Ng, and Xipeng Qiu. Hermes: Kv cache as hierarchical memory for efficient streaming video understanding. *arXiv preprint arXiv:2601.14724*, 2026.

[16] Ruyi Xu, Guangxuan Xiao, Yukang Chen, Liuning He, Kelly Peng, Yao Lu, and Song Han. Streamingvlm: Real-time understanding for infinite video streams. *arXiv preprint arXiv:2510.09608*, 2025.

[17] Zijia Zhao, Haoyu Lu, Yuqi Huo, Yifan Du, Tongtian Yue, Longteng Guo, Bingning Wang, Weipeng Chen, and Jing Liu. Needle in a video haystack: A scalable synthetic framework for benchmarking video mllms. *arXiv preprint arXiv:2406.09367*, 2(8), 2024.

[18] Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. Retrieval head mechanistically explains long-context factuality. *arXiv preprint arXiv:2404.15574*, 2024.

[19] Vivien Cabannes, Charles Arnal, Wassim Bouaziz, Alice Yang, Francois Charton, and Julia Kempe. Iteration head: A mechanistic study of chain-of-thought. *Advances in Neural Information Processing Systems*, 37:109101–109122, 2024.

A Appendix: Dataset Setup

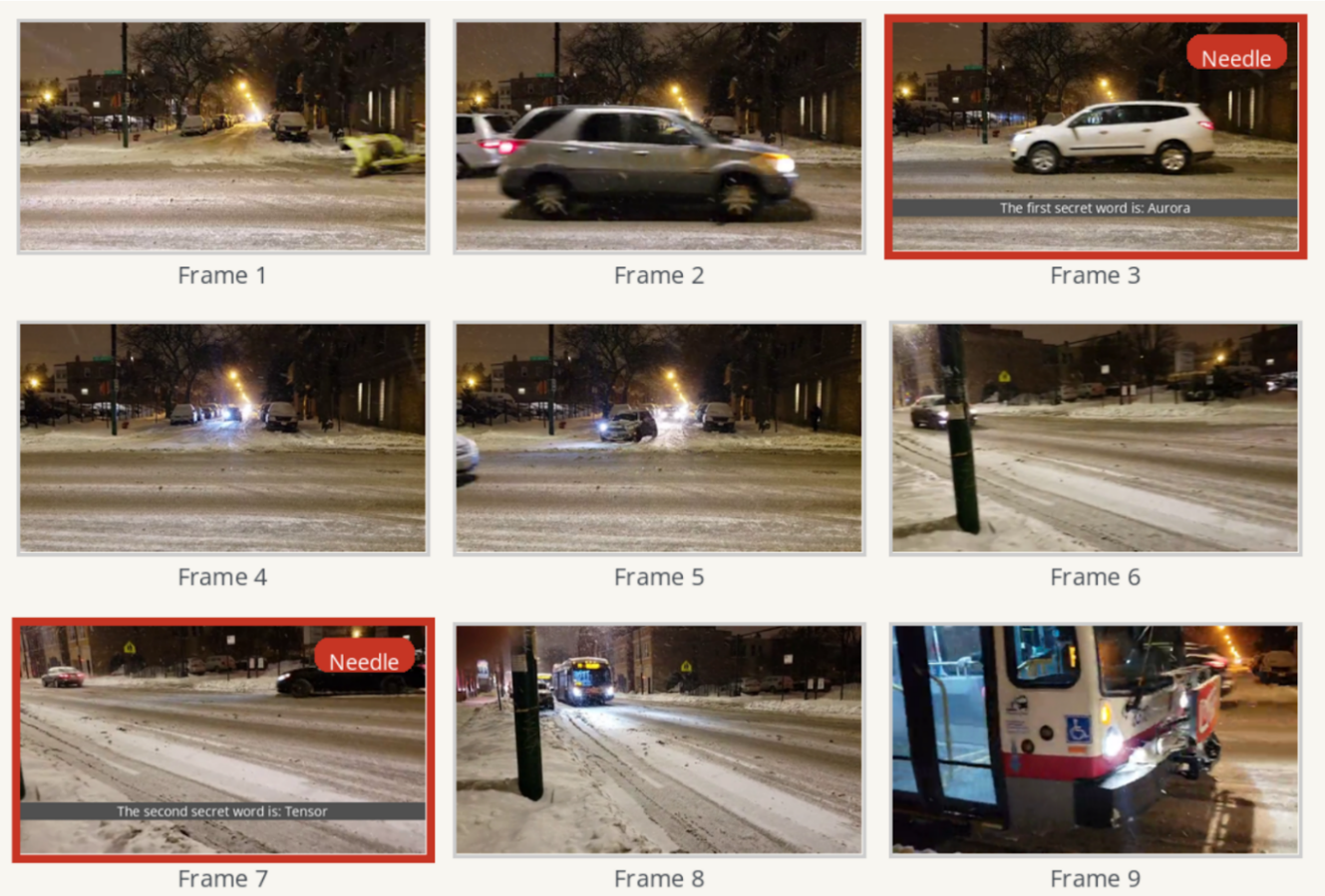

Figure 6: Synthetic retrieval example: 2 VNBench-style [17] needles inserted at random frames (3 and 7).

We used real videos from Pexels as background "haystack" videos; clips were typically around 2 minutes long. The generator overlays subtitle-style secret words onto a small number of sampled frames, creating a video needle-in-a-haystack retrieval task. Needle positions were randomized by default within 10-90% depth of the video, rather than fixed to preset frame indices.

B Appendix: Experiment Setup

For training, each sample used 64 uniformly sampled frames. We trained primarily with 5-needle retrieval. We also trained a 1-needle variant for comparison, but found that 5 needles gave better regularization and distillation loss.

Compute: Most experiments were run on a combination of **RTX 4090 and A6000 GPUs**. We also obtained access to **one H100** in the final week of the project, which was used for the demo.

Streaming Attention Details: We used **block-sparse attention** to implement streaming attention. During training, we used a sink size of 512 and a recent window of 1024. We briefly tested other combinations, but these choices did not appear to materially change training behavior. For inference, we used a default sink size of 256 and recent window of 512. This is because, in LLaVA-OneVision, one frame corresponds to roughly 196 tokens, so a sink of 256 covers about 1-2 frames and a recent window of 512 covers about 2-3 frames, which we found to be a reasonable default.